DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Projet JAVA – Etape 2

Table des matières

Etape 2 : Découverte des classes DAO 2
Etape 2.1 : Exécutions des tests unitaires d'origine 2
Etape 2.1.1 : Exécution : TestDaoEtape.java 2
Etape 2.1.2 : Exécution : TestDaoMiniExcursion.java 4
Etape 2.1.3 : Exécution : TestMiniExcursion.java 5
Etape 2.2 : Ajout de la méthode « getOneById() » 7
Bilan 10

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Etape 2 : Découverte des classes DAO

Etape 2.1 : Exécutions des tests unitaires d'origine

Etape 2.1.1 : Exécution : TestDaoEtape.java

Tout d'abord, on décide de tester les classes unitaires présentes dans le paquetage « test.dao ». On commence par exécuter la classe « TestDaoEtape » et nous obtenons le résultat ci-dessous. On apprend que 3 tests sont effectués dans cette classe lors de son exécution et qu'il concerne la deuxième excursion « codeExcursion = « E02 » ».

```
 Test 1 : TestDaoEtape.getOneById
mars 31, 2025 12:03:00 PM modele.dao.ConnexionBDD getConnexion
INFO: Connexion r►ussie ► la BDD : Thali
Etape d'id ("E02",1) trouv►e :
Etape{numEtape=1, description=Trajet aller en car, dureePrevue=45}

 Test 2 : TestDaoEtape.getAllByExcursion
Code de l'excursion concern►e : E02
Etape{numEtape=1, description=Trajet aller en car, dureePrevue=45}
Etape{numEtape=2, description=Randonn►e : aller au cap par les bois, dureePrevue=140}
Etape{numEtape=3, description=Pique-nique, dureePrevue=60}
Etape{numEtape=4, description=Randonn►e : retour du cap par la lande, dureePrevue=120}
Etape{numEtape=5, description=Trajet retour en car, dureePrevue=45}
Etape{numEtape=6, description=D►briefing, dureePrevue=30}
Etape{numEtape=7, description=D►briefing, dureePrevue=30}
Etape{numEtape=8, description=D►briefing, dureePrevue=30}
Etape{numEtape=9, description=D►briefing, dureePrevue=30}
Etape{numEtape=10, description=D►briefing, dureePrevue=30}
10 ►tapes trouv►es

 Test 3 : TestDaoEtape.insertFin
Etape lue : Etape{numEtape=11, description=D►briefing, dureePrevue=30}

Connexion ► la BDD ferm►e
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 1 : TestDaoEtape.java : Résultat obtenu après exécution de la classe. (« numEtape » : 2)

Le premier test consiste à afficher des informations en fonction de la méthode « getOneById() ». On peut observer que la description de l'étape 2 est affichée. Si le « numEtape » est différent alors une description différente est affichée. En l'occurrence, ici la valeur de « numEtape » est 1 donc la description affichée est la suivante (voir capture d'écran 3).

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

```
33        // Test 1 getOneById
34        System.out.println("\n Test 1 : TestDaoEtape.getOneById");
35        try {
36            codeExcursion = "E02";
37            numEtape = 1;
38            Etape etp = DaoEtape.getOneById(codeExcursion, numEtape);
```

Figure 2 : TestDaoEtape.java : Modification de la ligne 37. Chnagement de la valeur : 2 -> 1.

```
 Test 1 : TestDaoEtape.getOneById
mars 31, 2025 11:47:01 AM modele.dao.ConnexionBDD getConnexion
INFO: Connexion r►ussie ► la BDD : Thali
Etape d'id ("E02",1) trouv►e :
Etape{numEtape=1, description=Trajet aller en car, dureePrevue=45}
```

Figure 3 : TestDaoEtape.java : Résultat obtenu après exécution de la classe. (« numEtape » : 1)

Ensuite, le deuxième test affiche toutes les étapes de l'excursion en parcourant l'Arraylist « lesEtapes ». Les 10 étapes de l'excursion numéro 2 sont affichées.

Enfin, le troisième test ajoute une dernière étape finale à l'excursion actuelle (« E02 ») par le biais de la méthode de classe « insertFin() » présente dans la classe « DaoEtape ». A savoir, qu'étant donné que cette méthode est appelée à chaque exécution de la classe de test, un nouveau « Débrief » est ajouté à l'excursion « E02 ». Il faudrait donc empêcher ceci. (Le code corrigé n'a pas été rédigé, et n'a jamais été demandé). A la fin de ce programme, la connexion à la BDD est coupée.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Etape 2.1.2 : Exécution : TestDaoMiniExcursion.java

Le rôle de cette classe de test est de répertorier toutes les excursions, qu'elles soient gratuites ou payantes, tout en affichant la description de chaque étape et de chaque excursion. » Ces excursions sont récupérées par la méthode de classe getAll() de la classe « DaoMiniExcursion » puis leurs étapes sont affichées grâce à une boucle FOR parcourant l'Arraylist « lesExcursions ». La fin de ce programme se conclut par le nombre d'excursion trouvé et la fin de la connexion à la BDD.

```
 Test 1 : TestDaoMiniExcursion.getAll
mars 31, 2025 12:12:24 PM modele.dao.ConnexionBDD getConnexion
INFO: Connexion r►ussie ► la BDD : Thali
MiniExcursion{code=E01, libelle=Excursion dans l'►le, nbPlaces=8
        les ►tapes=
        1 : Etape{numEtape=1, description=travers►e aller, dureePrevue=30}
        2 : Etape{numEtape=2, description=promenade dans l'►le, dureePrevue=60}
        3 : Etape{numEtape=3, description=visite de la chapelle, dureePrevue=20}
        4 : Etape{numEtape=4, description=visite du phare, dureePrevue=30}
        5 : Etape{numEtape=5, description=promenade sur une petite crique, dureePrevue=15}
        6 : Etape{numEtape=6, description=visite du jardin exotique, dureePrevue=45}
        7 : Etape{numEtape=7, description=travers►e retour, dureePrevue=30} }
MiniExcursionPayante{MiniExcursion{code=E02, libelle=Randonn►e p►destre au "Cap des vents", nbPlaces=35
        les ►tapes=
        1 : Etape{numEtape=1, description=Trajet aller en car, dureePrevue=45}
        2 : Etape{numEtape=2, description=Randonn►e : aller au cap par les bois, dureePrevue=140}
        3 : Etape{numEtape=3, description=Pique-nique, dureePrevue=60}
        4 : Etape{numEtape=4, description=Randonn►e : retour du cap par la lande, dureePrevue=120}
        5 : Etape{numEtape=5, description=Trajet retour en car, dureePrevue=45}
        6 : Etape{numEtape=6, description=D►briefing, dureePrevue=30}
        7 : Etape{numEtape=7, description=D►briefing, dureePrevue=30}
        8 : Etape{numEtape=8, description=D►briefing, dureePrevue=30}
        9 : Etape{numEtape=9, description=D►briefing, dureePrevue=30}
        10 : Etape{numEtape=10, description=D►briefing, dureePrevue=30}
        11 : Etape{numEtape=11, description=D►briefing, dureePrevue=30} }
        tarif=25.75}
2 excursions trouv►es

Connexion ► la BDD ferm►e
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 4 : TestDaoMiniExcursion.java : Résultat obtenu après exécution de la classe.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Etape 2.1.3 : Exécution : TestMiniExcursion.java

Le rôle de cette classe de test est de vérifier si les ajouts et instanciations des excursions fonctionnent correctement. En comparant le code et le résultat en sortie dans la console, on observe une sorte d'incohérence. En effet, les tests effectués sur chaque étape (E01 et E02) affichent le même message. Ce message provient de la méthode toString() de la classe « MiniExcursion ». En vérifiant, la Javadoc ainsi que le code de cette classe, on trouve une méthode nommée « toStringEtat() » qui permettrait d'indiquer toutes les étapes ajoutées lors de l'exécution de cette classe. Il serait donc plus logique de l'utiliser pour les tests numéro 2 et 4 puisque leur intitulé est « ajout d'étapes ».

```
Test n►1 : instanciation mini excursion
E01 - Excursion dans l'►le - 8 pl.

Test n►2 : ajout d'►tapes
E01 - Excursion dans l'►le - 8 pl.

Test n►3 : instanciation mini excursion payante
E02 - Randonn►e p►destre au "Cap des vents" - 35 pl.

Test n►4 : ajout d'►tapes
E02 - Randonn►e p►destre au "Cap des vents" - 35 pl.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 5 : TestMiniExcursion.java : Résultat obtenu après exécution de la classe.

De ce fait, on obtiendrait le code et le résultat suivant :

```
27        me.ajouterEtape(new Etape(6, "visite du jardin exotique", 45));
28        me.ajouterEtape(new Etape(7, "traversée retour",  30));
29        System.out.println(me.toStringEtat()); // anciennement toString()
```

Figure 6 : TestMiniExcursion.java : Modification de la ligne 29. Remplacement de la méthode toString() par toStringEtat().

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

```
39        mep.ajouterEtape(new Etape(4, "Randonnée : retour du cap par la lande", 120 ));
40        mep.ajouterEtape(new Etape(5, "Trajet retour en car", 45));
41        System.out.println(mep.toStringEtat()); // anciennement toString()
```

Figure 7 : TestMiniExcursion.java : Modification de la ligne 29. Remplacement de la méthode toString() par toStringEtat().

```
Test n►1 : instanciation mini excursion
E01 - Excursion dans l'►le - 8 pl.

Test n►2 : ajout d'►tapes
MiniExcursion{code=E01, libelle=Excursion dans l'►le, nbPlaces=8
        les ►tapes=
        1 : Etape{numEtape=1, description=travers►e aller, dureePrevue=30}
        2 : Etape{numEtape=2, description=promenade dans l►►le, dureePrevue=60}
        3 : Etape{numEtape=3, description=visite de la chapelle, dureePrevue=20}
        4 : Etape{numEtape=4, description=visite du phare, dureePrevue=30}
        5 : Etape{numEtape=5, description=promenade sur une petite crique, dureePrevue=15}
        6 : Etape{numEtape=6, description=visite du jardin exotique, dureePrevue=45}
        7 : Etape{numEtape=7, description=travers►e retour, dureePrevue=30} }

Test n►3 : instanciation mini excursion payante
E02 - Randonn►e p►destre au "Cap des vents" - 35 pl.

Test n►4 : ajout d'►tapes
MiniExcursionPayante{MiniExcursion{code=E02, libelle=Randonn►e p►destre au "Cap des vents", nbPlaces=35
        les ►tapes=
        1 : Etape{numEtape=1, description=Trajet aller en car, dureePrevue=45}
        2 : Etape{numEtape=2, description=Randonn►e : aller au cap par les bois, dureePrevue=140}
        3 : Etape{numEtape=3, description=Pique-nique, dureePrevue=60}
        4 : Etape{numEtape=4, description=Randonn►e : retour du cap par la lande, dureePrevue=120}
        5 : Etape{numEtape=5, description=Trajet retour en car, dureePrevue=45} }
        tarif=25.0}
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 8 : TestMiniExcursion.java : Résultat obtenu après l'exécution du programme modifié.

Le code est resté dans le même état qu'après sa modification.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Etape 2.2 : Ajout de la méthode « getOneById() »

Tout d'abord, la méthode « getOneById » récupère une excursion spécifique à partir de son identifiant (codeExcursion). Pour cela, elle établit une connexion avec la base de données et exécute une requête SQL sécurisée (SELECT * FROM excursion WHERE Code = ?). En analysant son fonctionnement, on remarque qu'elle utilise un PreparedStatement pour éviter les injections SQL et affecte dynamiquement la valeur du paramètre attendu.

Lors de l'exécution, si une excursion correspondant à l'identifiant fourni est trouvée, ses informations (code, libellé et nombre de places) sont extraites du ResultSet et utilisées pour instancier un objet MiniExcursion. En vérifiant la gestion des ressources, on constate que la méthode utilise un **bloc try-with-resources**, garantissant ainsi la fermeture automatique de la connexion, de la requête préparée et du résultat obtenu.

En cas d'absence de correspondance dans la base, la méthode retourne null. Ce comportement permet d'éviter des erreurs en aval lors de l'utilisation de l'objet MiniExcursion, tout en offrant un moyen simple de vérifier si une excursion existe ou non.

```java
    /**
     * Méthode permettant de récupérer une MiniExcursion à partir de son code (ID)
     *
     * @param codeExcursion identifiant de l'excursion
     * @return liste d'objets de type MiniExcursion
     * @throws SQLException
     */
    public static MiniExcursion getOneById(String codeExcursion) throws SQLException {
        // Initialisation de l'objet MiniExcursion qui contiendra le résultat
        MiniExcursion miniExcurtionTrouvee = null;

        // Définition de la requête SQL avec un paramètre (le code de l'excursion)
        String query = "SELECT * FROM excursion WHERE Code = ?";

        // Utilisation d'un bloc try-with-resources pour gérer automatiquement la fermeture des ressources
        try (Connection cnx = ConnexionBDD.getConnexion(); // Connexion à la base de données
             PreparedStatement pstmt = cnx.prepareStatement(query)) { // Préparation de la requête SQL

            // Affectation du paramètre à la requête (remplacement du ? par la valeur du codeExcursion)
            pstmt.setString(1, codeExcursion);

            // Exécution de la requête et récupération des résultats dans un ResultSet
            try (ResultSet rs = pstmt.executeQuery()) {
                // Vérification si un résultat a été trouvé
                if (rs.next()) {
                    // Création de l'objet MiniExcursion avec les valeurs récupérées depuis la base
                    miniExcurtionTrouvee = new MiniExcursion(
                        rs.getString("Code"), // Récupération du code de l'excursion
                        rs.getString("libelle"), // Récupération du libellé de l'excursion
                        rs.getInt("NbPlaces")  // Récupération du nombre de places disponibles
                    );
                }
            }
        }
        // Retourne l'excursion trouvée (ou null si aucune excursion ne correspond au code donné)
        return miniExcurtionTrouvee;
    }
}
```

Figure 9 : DaoMiniExcursion.java : Création de la méthode « getOneById ».

Ce test a pour objectif de vérifier le bon fonctionnement de la méthode getOneById de la classe DaoMiniExcursion. En lui fournissant le code "E02", il tente de récupérer une excursion depuis la base de données. Si une excursion correspondante est trouvée, ses informations sont affichées ; sinon, un message indique l'absence de résultat. En cas d'erreur SQL, un message d'erreur est retourné, permettant d'identifier d'éventuels problèmes liés à la requête ou à la connexion.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

```java
35
36          // Test 2 getOneById
37          System.out.println("\n Test 2 : TestDaoMiniExcursion.getOneById");
38          String codeTest = "E02";
39          try {
40              MiniExcursion excursion = DaoMiniExcursion.getOneById(codeTest);
41              if (excursion != null) {
42                  System.out.println("Excursion trouvée : " + excursion);
43              } else {
44                  System.out.println("Aucune excursion trouvée pour le code " + codeTest);
45              }
46          } catch (SQLException ex) {
47              System.err.println("Erreur SQL lors du test getOneById : " + ex.getMessage());
48          }
49
```

Figure 10 : TestDaoMiniExcursion.java : test de la méthode « getOneById » de la classe « DaoMiniExcurston.java ».

Enfin, le code permettant la fermeture de la base de données est déplacé à la fin du programme de test.

```java
50          // Fermeture de la connexion
51              try {
52                  ConnexionBDD.getConnexion().close();
53                  System.out.println("\nConnexion à la BDD fermée");
54              } catch (SQLException ex) {
55                  Logger.getLogger(TestDaoMiniExcursion.class.getName()).log(Level.SEVERE, "TestDaoMiniExcursion - échec de la fermetu
56              }
57
58          }
```

Figure 11: TestDaoMiniExcursion.java : Code de fermeture de la connexion à la base de données.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Output - JavaApplication_1SLAM_PROJET2_THALI_MINI (run)

```
run:

 Test 1 : TestDaoMiniExcursion.getAll
mars 31, 2025 10:31:34 PM modele.dao.ConnexionBDD getConnexion
INFO: Connexion r►ussie ► la BDD : Thali
MiniExcursion{code=E01, libelle=Excursion dans l'►le, nbPlaces=8
        les ►tapes=
        1 : Etape{numEtape=1, description=travers►e aller, dureePrevue=30}
        2 : Etape{numEtape=2, description=promenade dans l'►le, dureePrevue=60}
        3 : Etape{numEtape=3, description=visite de la chapelle, dureePrevue=20}
        4 : Etape{numEtape=4, description=visite du phare, dureePrevue=30}
        5 : Etape{numEtape=5, description=promenade sur une petite crique, dureePrevue=15}
        6 : Etape{numEtape=6, description=visite du jardin exotique, dureePrevue=45}
        7 : Etape{numEtape=7, description=travers►e retour, dureePrevue=30} }
MiniExcursionPayante{MiniExcursion{code=E02, libelle=Randonn►e p►destre au "Cap des vents", nbPlaces=35
        les ►tapes=
        1 : Etape{numEtape=1, description=Trajet aller en car, dureePrevue=45}
        2 : Etape{numEtape=2, description=Randonn►e : aller au cap par les bois, dureePrevue=140}
        3 : Etape{numEtape=3, description=Pique-nique, dureePrevue=60}
        4 : Etape{numEtape=4, description=Randonn►e : retour du cap par la lande, dureePrevue=120}
        5 : Etape{numEtape=5, description=Trajet retour en car, dureePrevue=45}
        6 : Etape{numEtape=6, description=D►briefing, dureePrevue=30}
        7 : Etape{numEtape=7, description=D►briefing, dureePrevue=30}
        8 : Etape{numEtape=8, description=D►briefing, dureePrevue=30}
        9 : Etape{numEtape=9, description=D►briefing, dureePrevue=30}
        10 : Etape{numEtape=10, description=D►briefing, dureePrevue=30}
        11 : Etape{numEtape=11, description=D►briefing, dureePrevue=30}
        12 : Etape{numEtape=12, description=D►briefing, dureePrevue=30}
        13 : Etape{numEtape=13, description=D►briefing, dureePrevue=30}
        14 : Etape{numEtape=14, description=D►briefing, dureePrevue=30} }
        tarif=25.75}
2 excursions trouv►es

 Test 2 : TestDaoMiniExcursion.getOneById
Excursion trouv►e : E02 - Randonn►e p►destre au "Cap des vents" - 35 pl.

Connexion ► la BDD ferm►e
BUILD SUCCESSFUL (total time: 1 second)
```

Figure 12: TestDaoMiniExcursion.java : Résultat en sortie après l'exécution de la classe.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Bilan

Les étapes ont toutes été réalisées.

Problèmes rencontrés :

1) Les étapes n'étaient pas toutes affichées pour on se sait quelle raison.

```
 Test 1 : TestDaoEtape.getOneById
mars 31, 2025 11:32:27 AM modele.dao.ConnexionBDD getConnexion
INFO: Connexion r►ussie ► la BDD : Thali
Etape d'id ("E02",2) trouv►e :
Etape{numEtape=2, description=Randonn►e : aller au cap par les bois, dureePrevue=140}

 Test 2 : TestDaoEtape.getAllByExcursion
Code de l'excursion concern►e : E02
Etape{numEtape=1, description=Trajet aller en car, dureePrevue=45}
Etape{numEtape=2, description=Randonn►e : aller au cap par les bois, dureePrevue=140}
Etape{numEtape=3, description=Pique-nique, dureePrevue=60}
Etape{numEtape=4, description=Randonn►e : retour du cap par la lande, dureePrevue=120}
Etape{numEtape=5, description=Trajet retour en car, dureePrevue=45}
5 ►tapes trouv►es

 Test 3 : TestDaoEtape.insertFin
Etape lue : Etape{numEtape=6, description=D►briefing, dureePrevue=30}

Connexion ► la BDD ferm►e
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 13 : TestDaoEtape.java : Affichage incorrect (aucune modification n'avait été effectuée).

2) Lors des tests de la méthode « getOneById », la connexion à la base de données s'opérait. Il a donc fallut arranger le code de manière à ne plus avoir aucun soucis à ce niveau là.